Estación Meteorológica IoT

Ana Laura Tschen 221645



¿A qué capa pertenece JSON/SOAP según el modelo OSI y por qué?

Se asocian principalmente con la capa 7, aplicación, porque permiten representar e intercambiar información entre aplicaciones. JSON es un formato de datos y SOAP es un protocolo de mensajería. La representación y codificación de los datos también se relaciona conceptualmente con la capa 6,
presentación.

¿Qué beneficios tiene utilizar un formato como JSON/SOAP?

Permite estructurar los datos de manera consistente, facilita la comunicación entre sistemas desarrollados en distintos lenguajes y hace más sencillo interpretar, validar y mantener los mensajes. JSON es especialmente fácil de leer y procesar, aunque ocupa más espacio que una codificación binaria
compacta.

Envío de datos del productor

```
(dm1) PS D:\estacion_iot> python producer.py --mode binary --interval 20
Productor: topic=221645, modo=binary, offline=False
{'temperatura': 33.71, 'humedad': 13, 'direccion_viento': 'N'} | payload=3 bytes | partition=1, offset=0
{'temperatura': 68.94, 'humedad': 56, 'direccion_viento': 'NO'} | payload=3 bytes | partition=1, offset=1
{'temperatura': 39.94, 'humedad': 66, 'direccion_viento': 'N'} | payload=3 bytes | partition=1, offset=2
{'temperatura': 36.69, 'humedad': 85, 'direccion_viento': 'NE'} | payload=3 bytes | partition=1, offset=3
{'temperatura': 45.63, 'humedad': 70, 'direccion_viento': 'O'} | payload=3 bytes | partition=1, offset=4
{'temperatura': 48.85, 'humedad': 84, 'direccion_viento': 'SO'} | payload=3 bytes | partition=1, offset=5
{'temperatura': 55.16, 'humedad': 46, 'direccion_viento': 'S'} | payload=3 bytes | partition=1, offset=6
{'temperatura': 41.16, 'humedad': 62, 'direccion_viento': 'S'} | payload=3 bytes | partition=1, offset=7
{'temperatura': 73.52, 'humedad': 32, 'direccion_viento': 'SO'} | payload=3 bytes | partition=1, offset=8
{'temperatura': 41.68, 'humedad': 81, 'direccion_viento': 'SO'} | payload=3 bytes | partition=1, offset=9
{'temperatura': 30.23, 'humedad': 60, 'direccion_viento': 'S'} | payload=3 bytes | partition=1, offset=10
{'temperatura': 27.35, 'humedad': 55, 'direccion_viento': 'NO'} | payload=3 bytes | partition=1, offset=11
{'temperatura': 62.33, 'humedad': 56, 'direccion_viento': 'E'} | payload=3 bytes | partition=1, offset=12
{'temperatura': 70.38, 'humedad': 19, 'direccion_viento': 'O'} | payload=3 bytes | partition=1, offset=13
{'temperatura': 33.38, 'humedad': 64, 'direccion_viento': 'SO'} | payload=3 bytes | partition=1, offset=14
{'temperatura': 60.26, 'humedad': 60, 'direccion_viento': 'NE'} | payload=3 bytes | partition=1, offset=15
{'temperatura': 70.27, 'humedad': 49, 'direccion_viento': 'NO'} | payload=3 bytes | partition=1, offset=16
{'temperatura': 26.8, 'humedad': 81, 'direccion_viento': 'N'} | payload=3 bytes | partition=1, offset=17
{'temperatura': 47.08, 'humedad': 39, 'direccion_viento': 'SE'} | payload=3 bytes | partition=1, offset=18
{'temperatura': 51.95, 'humedad': 81, 'direccion_viento': 'SE'} | payload=3 bytes | partition=1, offset=19
```

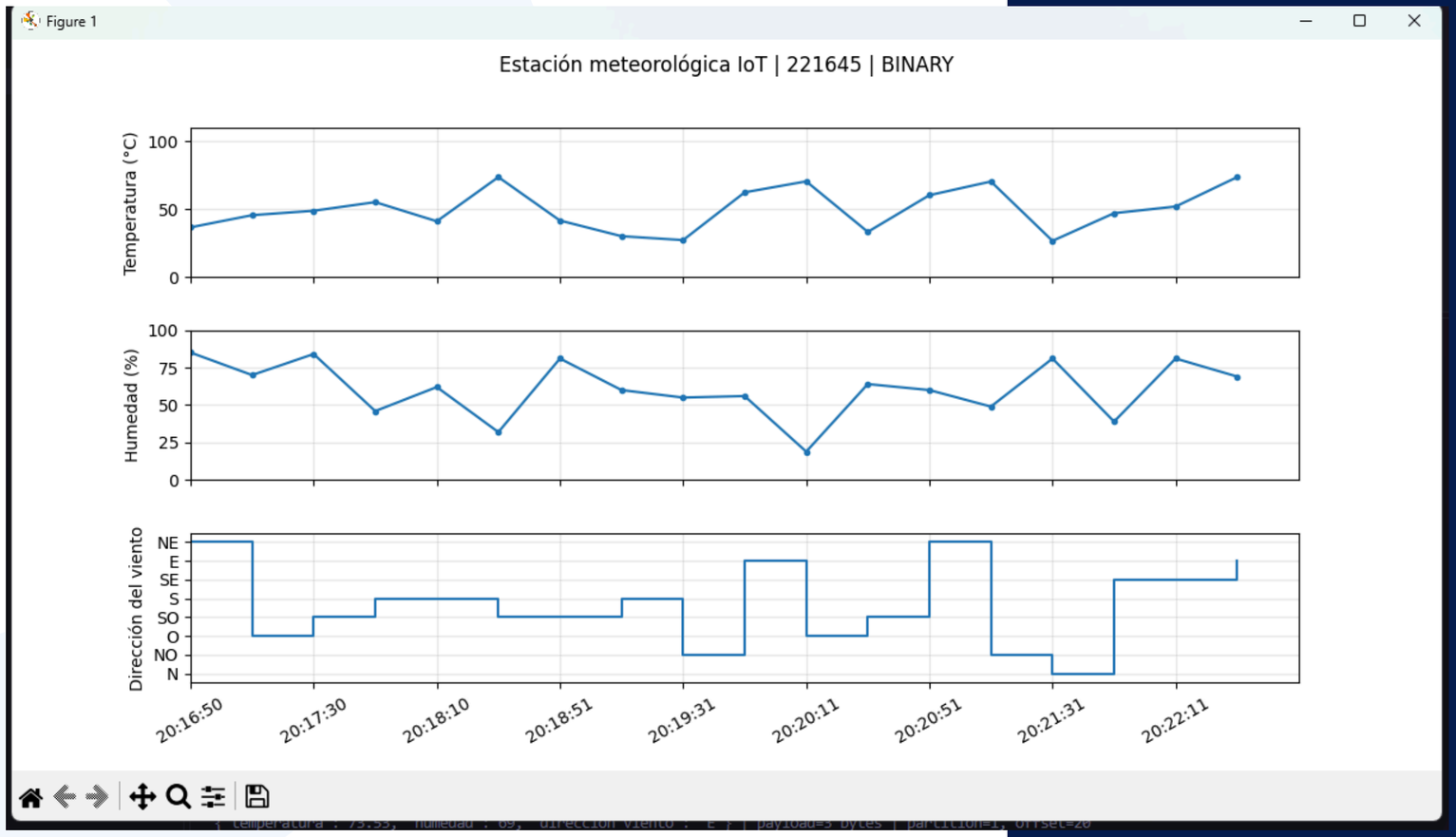
El productor envía una medición cada 20 segundos a un topic propio (221645) usando Apache Kafka y autenticación SASL/PLAIN

Consumo y visualización

```
(dm1) PS D:\estacion_iot> python consumer.py --mode binary
Escuchando 221645 (binary)...
20:16:50 | {'temperatura': 36.69, 'humedad': 85, 'direccion_viento': 'NE'} | muestras=1
20:17:10 | {'temperatura': 45.63, 'humedad': 70, 'direccion_viento': 'O'} | muestras=2
20:17:30 | {'temperatura': 48.85, 'humedad': 84, 'direccion_viento': 'SO'} | muestras=3
20:17:50 | {'temperatura': 55.16, 'humedad': 46, 'direccion_viento': 'S'} | muestras=4
20:18:10 | {'temperatura': 41.16, 'humedad': 62, 'direccion_viento': 'S'} | muestras=5
20:18:31 | {'temperatura': 73.52, 'humedad': 32, 'direccion_viento': 'SO'} | muestras=6
20:18:51 | {'temperatura': 41.68, 'humedad': 81, 'direccion_viento': 'SO'} | muestras=7
20:19:11 | {'temperatura': 30.23, 'humedad': 60, 'direccion_viento': 'S'} | muestras=8
20:19:31 | {'temperatura': 27.35, 'humedad': 55, 'direccion_viento': 'NO'} | muestras=9
20:19:51 | {'temperatura': 62.33, 'humedad': 56, 'direccion_viento': 'E'} | muestras=10
20:20:11 | {'temperatura': 70.38, 'humedad': 19, 'direccion_viento': 'O'} | muestras=11
20:20:31 | {'temperatura': 33.38, 'humedad': 64, 'direccion_viento': 'SO'} | muestras=12
20:20:51 | {'temperatura': 60.26, 'humedad': 60, 'direccion_viento': 'NE'} | muestras=13
20:21:11 | {'temperatura': 70.27, 'humedad': 49, 'direccion_viento': 'NO'} | muestras=14
20:21:31 | {'temperatura': 26.8, 'humedad': 81, 'direccion_viento': 'N'} | muestras=15
20:21:51 | {'temperatura': 47.08, 'humedad': 39, 'direccion_viento': 'SE'} | muestras=16
```

El consumidor se suscribe al topic, decodifica cada medición y actualiza las gráficas de temperatura, humedad y direscción del viento conforme llegan nuevos mensajes

Consumo y visualización



¿Qué ventajas y desventajas considera que tiene este acercamiento basado en Pub/Sub de Kafka?

Entre las ventajas están la separación entre productor y consumidor, la posibilidad de conectar varios consumidores, procesar mensajes de manera independiente y escalar el sistema. También se pueden retener y re procesar eventos. Como desventajas, requiere configurar y mantener un broker, manejar autenticación, particiones y offsets, y puede resultar demasiado complejo para una aplicación
pequeña.

¿Para qué aplicaciones tiene sentido usar Kafka? ¿Para cuáles no?

Tiene sentido en sistemas que generan muchos eventos, como plataformas IoT con numerosos sensores, monitoreo en tiempo real, análisis de registros y procesamiento de transacciones. No suele ser la opción más conveniente para una aplicación sencilla con pocos mensajes, dispositivos con recursos limitados o comunicaciones que requieren una respuesta directa e inmediata.

Restricción de payload de 3 bytes

Se codifican los valores antes de enviarlos y se descodifican en el consumidor, la misma visualización funciona con mensajes binarios de exactamente 3 bytes.

Temperatura: 14 bits Humedad: 7 bits Viento: 3 bits

14 + 7 + 3 = 24 bits = 3 bytes

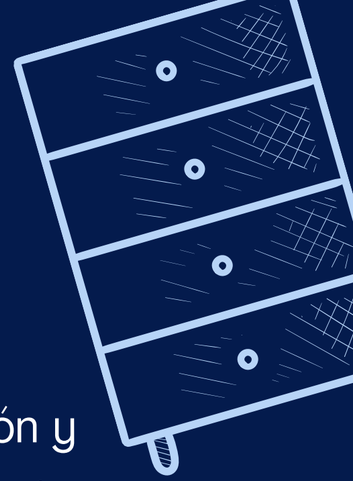
La temperatura se multiplica por 100: 110.00 °C → 11000 (cabe en 14 bits). Luego se divide entre 100 para recuperar los decimales.



Restricción de payload de 3 bytes

```
(dm1) PS D:\estacion_iot> python consumer.py --mode json
Modo offline: siguiendo telemetria.jsonl
(dm1) PS D:\estacion_iot> python consumer.py --mode binary
Modo offline: siguiendo telemetria.jsonl
15:58:51 | {'temperatura': 49.39, 'humedad': 56, 'direccion_viento': 'SO'} |
 muestras=1
15:59:06 | {'temperatura': 42.78, 'humedad': 65, 'direccion_viento': 'S'} |
muestras=2
15:59:21 | {'temperatura': 57.26, 'humedad': 46, 'direccion_viento': 'NO'} |
 muestras=3
```

```
(dm1) PS D:\estacion_iot> python producer.py --mode binary --interval 15 --c
ount 3
Productor: topic=221645, modo=binary, offline=True
{'temperatura': 49.39, 'humedad': 56, 'direccion_viento': 'SO'} | payload=3
bytes | guardado en telemetria.jsonl
{'temperatura': 42.78, 'humedad': 65, 'direccion_viento': 'S'} | payload=3 b
ytes | guardado en telemetria.jsonl
{'temperatura': 57.26, 'humedad': 46, 'direccion_viento': 'NO'} | payload=3
bytes | guardado en telemetria.jsonl
(dm1) PS D:\estacion_iot>
```



¿Qué complejidades introduce el tener un payload restringido (pequeño)?

Obliga a reducir la información transmitida, elegir cuidadosamente la precisión de cada medición y codificar los datos a nivel de bits. Además, el productor y el consumidor deben utilizar exactamente el mismo esquem

¿Cómo podemos hacer que el valor de temperatura quepa en 14 bits?

Multiplicamos la temperatura por 100 y la convertimos en un entero. Así, una temperatura máxima de 110.00 °C se convierte en 11000, un valor que cabe en 14 bits porque 2^14 = 16384. Al recibirla, dividimos entre 100 para recuperar los dos decimales. De esta forma utilizamos 14 bits para temperatura, 7 para humedad y 3 para dirección del viento: 24 bits en total, equivalentes a 3 bytes.



¿Qué sucedería si ahora la humedad también es tipo float con un decimal? ¿Qué decisiones tendríamos que tomar en ese caso?

Para representar valores de 0.0 % a 100.0 % con un decimal necesitaríamos 10 bits. Junto con los 14 bits de temperatura y los 3 bits del viento sumaríamos 27 bits, superando el límite. Una alternativa sería reducir la precisión de temperatura a un decimal: necesitaría 11 bits, y 11 + 10 + 3 = 24 bits. También podríamos reducir rangos, separar mediciones o aumentar el tamaño permitido del mensaje.

¿Qué parámetros o herramientas de Kafka podrían ayudarnos si las restricciones fueran aún más fuertes?

Kafka permite comprimir lotes de mensajes mediante compression_type y agrupar envíos ajustando batch_size y linger_ms, lo cual puede reducir el tráfico total. Pero estas opciones no garantizan que cada payload individual cumpla un límite de 3 bytes. Para ese caso necesitamos seguir usando un formato binario compacto o reducir la precisión y cantidad de datos enviados.